

Lab 3: Loggy - A Logical Time Logger

Sevastian Bajenov

September 22, 2026

1 Introduction

In this laboratory work, my task was to implement a simple logger and a network of workers, and investigate the concept of logical time within the topology. Logical clocks are used to represent partial order of the events in a system. The processes handle events by understanding the happened-before relation between them. To investigate this concept I created basic logger setup and then enhanced it with Lamport and vector clocks.

2 Main problems and solutions

The implementation of both Lamport and vector clocks is based on several generic methods. They are responsible for incrementing local counter and also updating the clock, merging the incoming timestamps with the current one, and ordering the events based on their timestamps. The biggest challenge for me in the time interface was to understand how to handle causal order in the vector clock setup because of the way the timestamps are stored.

At first I tested the setup where the logger did not order the messages at all. It is important to mention that each test adds two delays to the worker so that it doesn't talk permanently to other workers and that its message to the logger is late compared to the message to another worker. Therefore, many messages appeared out of order: the text of the messages showed that the "received" message was displayed earlier than the "send" one. Also the Lamport timestamps were mixed up.

The correct ordering of messages via queue can be achieved but we still will not be able to claim the order of two events based on their Lamport timestamps. However, we can be sure about the order of the events, knowing their background and counters. The following section presents more tests of the Lamport and vector clocks.

3 Evaluation

The process of evaluation of the logical clocks considers not the order but rather the efficiency of different implementations. I investigated the logs targeting message addition to the holdback queue and these are the results:

Lamport clock

```
log: 10 paul {sending,{hello,21}}
log: 10 john {sending,{hello,95}}
holdback: size 4
holdback: size 5
holdback: size 6
holdback: size 7
holdback: size 8
log: 11 john {sending,{hello,30}}
log: 11 paul {received,{hello,95}}
log: 11 ringo {received,{hello,21}}
holdback: size 6
log: 12 ringo {sending,{hello,86}}
log: 13 paul {received,{hello,86}}
log: 13 ringo {sending,{hello,43}}
holdback: size 4
holdback: size 5
```

Vector clock

```
log: [{john,9},{paul,9},{ringo,9},{george,5}] john {sending,{hello,69}}
log: [{george,8},{john,9},{ringo,9},{paul,9}] george {received,{hello,69}}
holdback: size 1
holdback: size 2
log: [{paul,10},{george,5},{john,6},{ringo,7}] paul {sending,{hello,70}}
log: [{ringo,11},{paul,10},{george,7},{john,6}] ringo {received,{hello,70}}
log: [{ringo,12},{paul,10},{george,7},{john,6}] ringo {sending,{hello,57}}
log: [{paul,11},{george,7},{john,6},{ringo,12}] paul {received,{hello,57}}
holdback: size 1
holdback: size 2
log: [{george,9},{john,9},{ringo,9},{paul,9}] george {sending,{hello,96}}
log: [{ringo,13},{paul,10},{george,9},{john,9}] ringo {received,{hello,96}}
holdback: size 1
```

Clearly the Lamport clock provides us with less causal information about the order of events and therefore the holdback queue stays more populated

for the purpose of synchronization. The test for the maximum number of entries in the holdback queue confirms it: Lamport clock logger reaches 19 entries and the vector one - 4.

The final test demonstrates that the vector clock helps the logger handle dynamic sets of workers efficiently even if the group under observation changes during the test run:

```
log: [{b,4},{a,6},{c,3}] b {sending,{hello,49}}
log: [{a,7},{c,3},{b,4}] a {received,{hello,49}}
log: [{a,8},{c,3},{b,4}] a {sending,{hello,24}}
holdback: size 1
log: [{b,5},{a,8},{c,3}] b {received,{hello,24}}
holdback: size 1
holdback: size 2
log: [{d,1}] d {sending,{hello,30}}
log: [{e,1},{d,1}] e {received,{hello,30}}
holdback: size 1
log: [{b,6},{a,8},{c,3}] b {sending,{hello,10}}
log: [{a,9},{c,3},{b,6}] a {received,{hello,10}}
holdback: size 1
log: [{c,4},{a,1}] c {sending,{hello,100}}
log: [{e,2},{d,1},{c,4},{a,1}] e {received,{hello,100}}
```

4 Conclusions

In this laboratory work I explored two ways of implementing logical time in a distributed system. Lamport clock demonstrated that it can offer us partial event order. In case the two processes have a causal relationship then their Lamport values will reflect that. However, inverse hypothesis does not work and we have no precise way to identify concurrent events. On the other hand, vector clocks give us the opportunity to understand the whole order of the events (concurrent ones as well), but still need an overhead because of the way process counters are stored - in a time vector.